Outnumbered Tales

Tales from the skewed side of the distribution.

Igor L.R. Azevedo • Mind Lab, Imperial College London

Tale *0*

Nature is Zipfian — and our data inherits the skew.

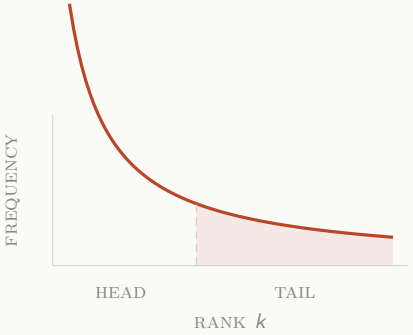
A handful of classes account for most of the examples; the rest fall into a long, thin tail of categories that appear only a few times each.

Zipf's Law

If we rank classes by frequency, the k-th class appears with probability

$$P(y_k) \propto k^{-s}, \quad s > 1$$

A few classes dominate; most fall into a long, thin tail.



From True Risk to Empirical Risk

The True Risk $R(f)$ is the expected loss over the unknown joint distribution $P(x, y)$:

$$R(f) = \mathbb{E}_{(x,y)\sim P}[\mathcal{L}(f(x), y)]$$

Since $P(x, y)$ is inaccessible, we approximate it with the Empirical Risk over a finite dataset $\mathcal{D}$:

$$\hat{R}(f) = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}(f(x_i),\, y_i)$$

Empirical Risk Minimisation (ERM) selects $\hat{f} = \arg\min_{f\in\mathcal{F}} \hat{R}(f)$. The Law of Large Numbers guarantees $\hat{R}_N(f) \xrightarrow{N\to\infty} R(f)$ a.s.

Gradient Starvation

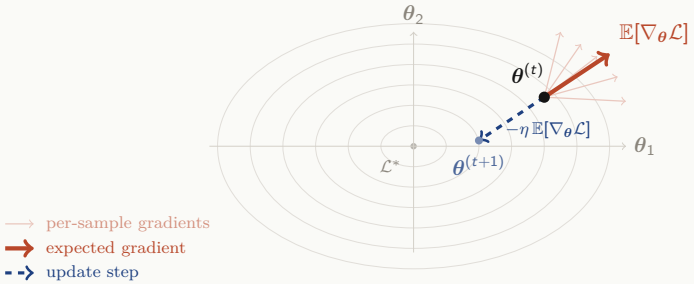
The expected gradient decomposes across classes:

$$\mathbb{E}[\nabla_{\theta}\mathcal{L}] = \sum_{c=1}^{d} P(\hat{y} = c) \cdot \mathbb{E}_{x|y=c}[\nabla_{\theta}\mathcal{L}(f_{\theta}(x), c)]$$

- $P(\hat{y} = c)$ acts as a scalar multiplier on each class's gradient contribution.
- Head classes have large $P(\hat{y}=c)$ and dominate the update direction.
- Tail classes have tiny $P(\hat{y}=c)$; even if their per-class gradient is large, it is scaled to near zero.
- Over many steps, tail classes remain starved of gradient signal.

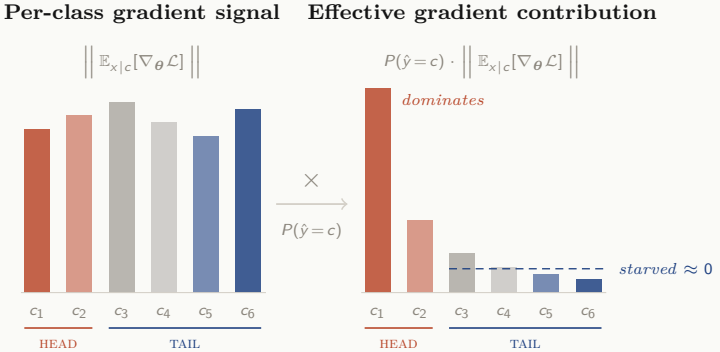
The Long-Tail Implication

$$\mathbb{E}[\nabla_\theta \mathcal{L}] = \underbrace{P(\hat{y} = c_{\text{head}})}_{\text{large}} \cdot \mathbb{E}_{x|c_{\text{head}}}[\nabla \mathcal{L}] \; + \; \underbrace{P(\hat{y} = c_{\text{tail}})}_{\text{tiny}} \cdot \mathbb{E}_{x|c_{\text{tail}}}[\nabla \mathcal{L}]$$



θ_1, θ_2: two representative dimensions of the model's full parameter vector θ.

Why Starvation Happens



All classes carry comparable intrinsic learning signal, but Zipf weighting crushes tail contributions to near zero. The optimiser is effectively blind to them.

Tale *1*

The Trap of Overall Accuracy.

A model can score 95% accuracy while failing on every class that matters most.

Key Metrics at a Glance

Metric	Formula	Imbalance issue
Accuracy	$\frac{\sum_c \mathrm{TP}_c}{N}$	Trivially high if majority always predicted
Recall_c	$\frac{\mathrm{TP}_c}{\mathrm{TP}_c+\mathrm{FN}_c}$	Near-zero on tail if model ignores rare classes
$\mathrm{Precision}_c$	$\frac{\mathrm{TP}_c}{\mathrm{TP}_c+\mathrm{FP}_c}$	High for head but misleading without recall
$F_{1,c}$	$2 \cdot \frac{\mathrm{Prec}_c \cdot \mathrm{Rec}_c}{\mathrm{Prec}_c+\mathrm{Rec}_c}$	Per-class; dominated by smaller value
Macro-F_1	$\frac{1}{d}\sum_c F_{1,c}$	Listens to the tail: equal class weight

AUROC vs AUPRC

AUROC: TPR vs FPR across all thresholds.

- Baseline = 0.5; perfect = 1.0.
- Under imbalance, FPR denominator (FP+TN) is huge, so many false positives barely move FPR — inflating the curve.

AUPRC: Precision vs Recall across all thresholds.

- Baseline = class prior $P(y=1)$.
- Every FP directly hurts Precision — far more sensitive to tail performance.
- Preferred when cost of missing a positive is high.

Tale *2*

Rebalancing the Scales.

If the training distribution misrepresents the world, the simplest remedy is to reshape the distribution itself.

The Re-Sampling Idea

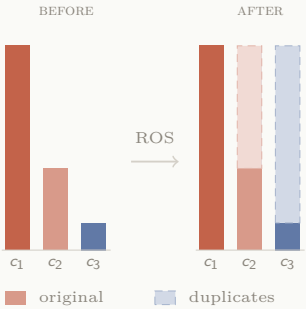
Rather than changing the loss or the architecture, re-sampling modifies $\mathcal{D}$ so that the empirical class prior better approximates a balanced distribution. It directly manipulates $P(\hat{y} = c)$ in the gradient decomposition, amplifying tail contributions.

- Over-sampling (ROS): duplicate minority examples. Risk: overfitting to exact duplicates.
- Under-sampling (RUS): discard majority examples. Risk: data loss on head classes.
- Synthetic data (SMOTE): generate new, plausible minority examples by interpolation [2].

Random Over-Sampling (ROS)

Duplicate randomly selected minority examples until every class has roughly the same count.

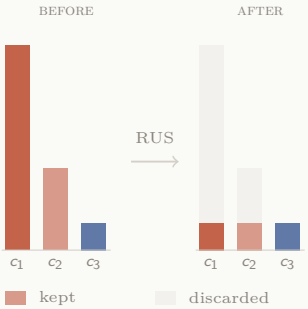
- Simple, requires no new information.
- Risk: exact duplicates can cause overfitting.
  - The model memorises individual tail examples rather than learning generalisable features.



Random Under-Sampling (RUS)

Discard randomly selected majority examples until all classes are roughly balanced.

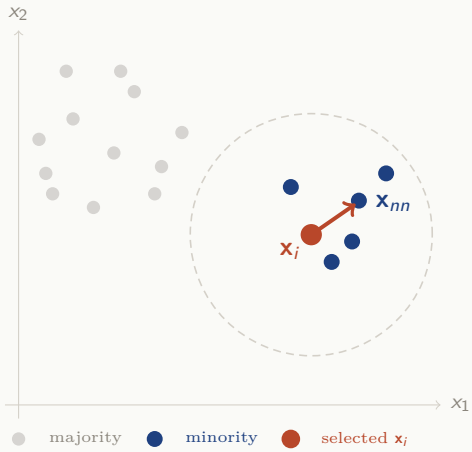
- Preserves only real data — no risk of memorising duplicates.
- Risk: discards potentially valuable majority-class information, reducing dataset size and degrading head-class performance.



SMOTE (1) Pick a Minority Sample and a Neighbor

Synthetic Minority Over-sampling [2] generates new examples by interpolation. For each minority sample $\mathbf{x}_i$, find its k nearest neighbors and select one $\mathbf{x}_{nn}$:

$$\mathbf{x}_{nn} \in \mathsf{kNN}(\mathbf{x}_i,\, k)$$



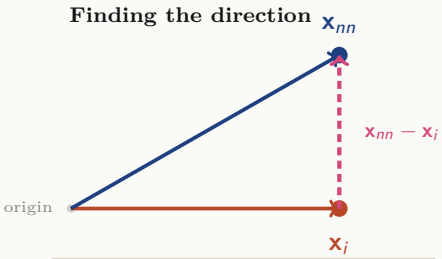
SMOTE (2) Interpolate Along the Segment

Compute the difference vector and sample a random scalar $\lambda \sim \text{Uniform}(0, 1)$.

The synthetic point is:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda\,(\mathbf{x}_{nn} - \mathbf{x}_i)$$

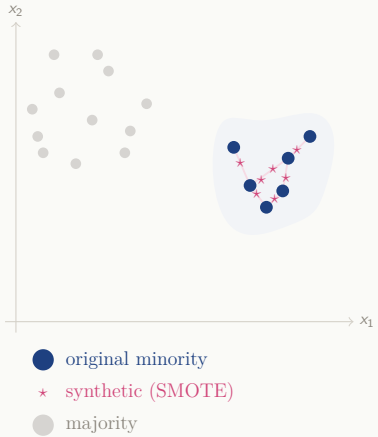
λ constrains $\mathbf{x}_{\text{new}}$ to the segment between $\mathbf{x}_i$ and $\mathbf{x}_{nn}$.



SMOTE (3) The Augmented Distribution

Repeat for multiple minority samples and neighbors until the desired class balance is achieved.

- SMOTE fills gaps between existing minority samples rather than duplicating points.
- Each synthetic sample is unique, reducing overfitting risk compared to ROS.
- Limitation: interpolation assumes the feature space between neighbors is class-consistent. Near decision boundaries, synthetic points may invade majority territory.



Tale *3*

Margins that Listen to the Tail.

Instead of changing the data, change the loss — give rare classes wider decision margins so the model cannot ignore them.

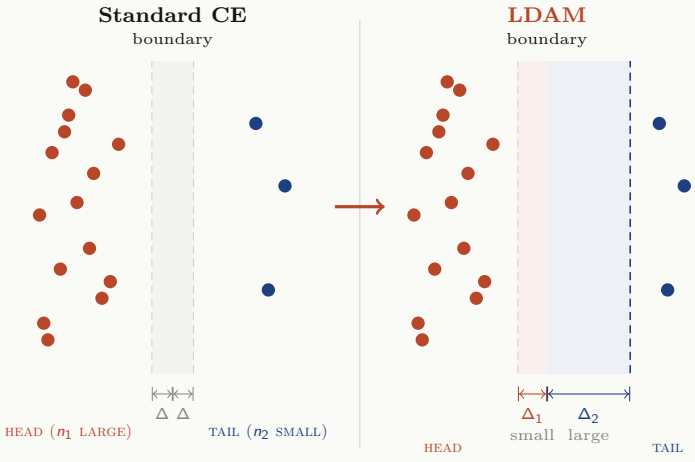
LDAM at a Glance

Label-Distribution Aware Margin Loss [1] enforces larger margins for classes with fewer training examples: $\Delta_c \propto n_c^{-1/4}$.

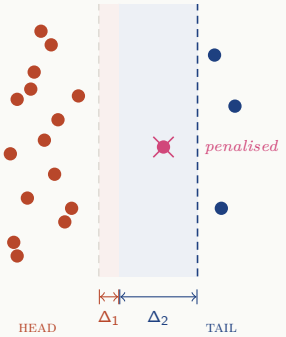
- Tail classes (small n_c) get large Δ_c — the model must push their logits further from the decision boundary.
- Head classes (large n_c) get small Δ_c — they already have enough data.
- Unlike re-sampling, LDAM acts entirely through the loss function, leaving the data untouched.

Key insight: standard training learns good representations even under imbalance. The poor performance stems from the classifier biasing its boundaries toward the majority. Re-sampling fixes the classifier but damages representation learning.

How LDAM Margins Work



Why the Wider Margin Helps the Tail



The wider margin Δ_2 creates a buffer zone around the decision boundary for tail classes.

- Any tail example that falls inside the margin zone is penalised, even if technically on the correct side.
- This forces the model to push tail logits far from the boundary.
- Head classes need only a narrow margin.

The LDAM Loss

LDAM subtracts a class-dependent margin Δ_y from the correct-class logit before the softmax:

$$\mathcal{L}_{\text{LDAM}} = -\log \frac{e^{f(x)_y - \Delta_y}}{e^{f(x)_y - \Delta_y} + \sum_{l \neq y} e^{f(x)_l}} \qquad \Delta_j = \frac{C}{n_j^{1/4}}$$

The model must produce a logit at least Δ_y higher than standard CE requires. For tail classes this penalty is harsh; for head classes it is negligible.

Deferred Re-Weighting (DRW)

Applying class weights from the start distorts gradients and damages representation learning. DRW uses a two-stage schedule:

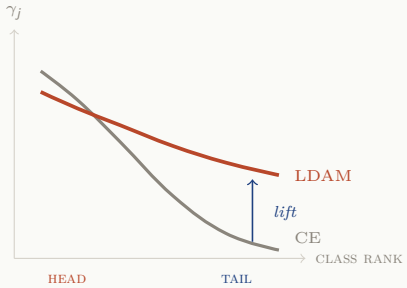
$$w_j^{(t)} = \begin{cases} 1 & \text{if } t < T \quad \text{(Stage 1: learn features)} \\ \dfrac{1 \,/\, n_j}{\sum_{l=1}^{k} 1 \,/\, n_l} & \text{if } t \geq T \quad \text{(Stage 2: fix classifier)} \end{cases}$$

The full LDAM-DRW loss at epoch t:

$$\mathcal{L}_{\text{LDAM-DRW}}^{(t)} = -\, w_y^{(t)} \, \log \frac{e^{f(x)_y - \Delta_y}}{e^{f(x)_y - \Delta_y} + \sum_{l \neq y} e^{f(x)_l}}$$

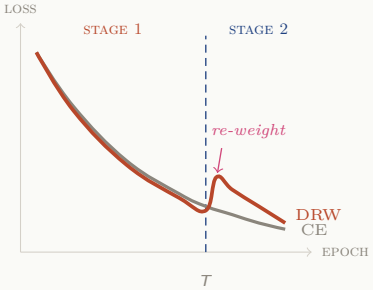
The Effect of LDAM-DRW

Logit gap by class



LDAM lifts the margin of tail classes, making predictions more confident.

Training loss over epochs



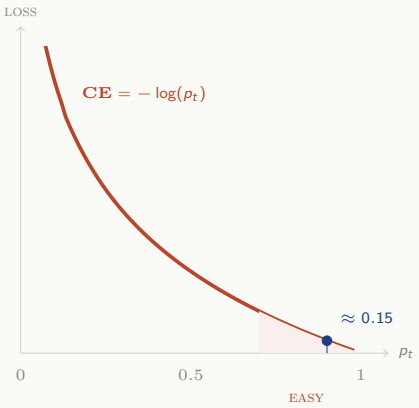
DRW introduces a brief bump at epoch T as the classifier re-adjusts to balanced weights.

Tale 4

Focusing on What's Hard.

Down-weight the easy examples and let the model spend its capacity on the ones it struggles with.

The Problem: Easy Examples Dominate



Let p_t be the model's probability for the correct class. Even when $p_t = 0.9$, CE loss ≈ 0.15 — not zero.

With 10k easy negatives ($\approx$0.15 each) and 50 hard positives ($\approx$1.74 each): easy examples produce 17× more total loss [6].

The gradient is overwhelmingly driven by examples the model already classifies correctly.

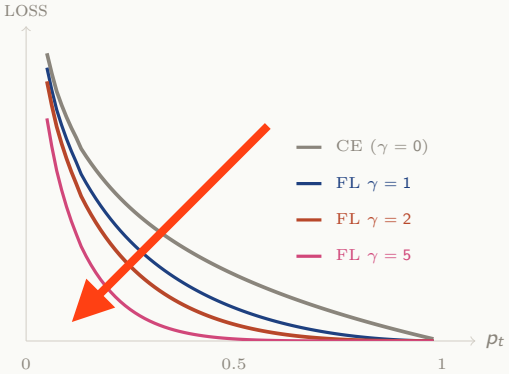
The Focal Loss

Add a modulating factor $(1 - p_t)^\gamma$ and a class-balancing weight α_t:

$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

- Hard example ($p_t = 0.1$): $(1 - p_t)^\gamma = 0.9^\gamma \approx 1$ — loss largely unaffected.
- Easy example ($p_t = 0.99$): $(1 - p_t)^\gamma = 0.01^\gamma \approx 0$ — loss squashed to near zero.
- α_t addresses class imbalance; $(1 - p_t)^\gamma$ addresses easy/hard imbalance.
- $\gamma = 0$ recovers standard α-balanced CE. In practice $\gamma = 2$ works well.

CE vs Focal Loss



As γ increases, the loss for well-classified examples ($p_t \to 1$) is increasingly suppressed, focusing training on the hard cases.

Tale 5

Adjusting the Logits.

If the training distribution is skewed, correct for it directly in the model's output scores.

Standard vs. Balanced Error

Menon et al. [7] distinguish two objectives:

Standard error:

$$L^{\text{std}} = \sum_y \pi_y \, P_{x|y}[\hat{y} \neq y]$$

Rare classes barely affect the sum.

Balanced error:

$$L^{\text{bal}} = \frac{1}{k} \sum_y P_{x|y}[\hat{y} \neq y]$$

Every class contributes equally.

The Bayes-optimal classifier differs: standard $\to \arg\max_y P(y \mid x)$; balanced $\to \arg\max_y P(y \mid x)/P(y) = \arg\max_y P(x \mid y)$. The only difference is whether the class prior $P(y)$ is left in or divided out.

The Prior Is the Problem

Standard CE trains logits toward $f_y^*(x) = \log P(y \mid x) + \text{const}$. Expanding via Bayes:

$$f_y^*(x) = \log P(x \mid y) + \log P(y) + \text{const}$$

- $\log P(y)$ is a built-in bias: head classes get a positive boost, tail classes get a negative penalty.
- For balanced error, the Bayes-optimal scorer needs only $\log P(x \mid y)$ — the prior must be removed.

Solution: subtract $\log P(y)$ from each logit, leaving $f_y^*(x) = \log P(x \mid y) + \text{const}$. This is Fisher consistent for balanced error (unlike LDAM or Equalization Loss).

Logit-Adjusted Cross-Entropy

Define adjusted logits $g_c(x) = f_c(x) + \log \pi_c$ and feed them into CE:

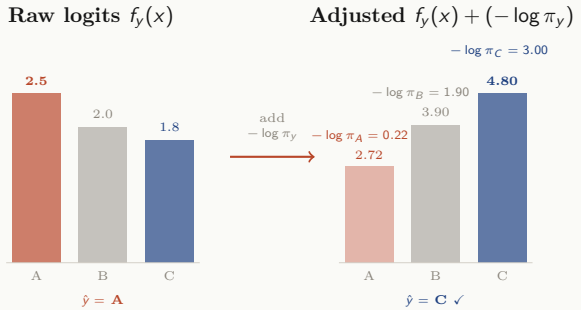
$$\mathcal{L}_{\mathsf{LA}}(y,\, f(x)) = \log\Big[1 + \sum_{y' \neq y} e^{\,f_{y'} - f_y \,+\, \log \pi_{y'} - \log \pi_y}\Big]$$

- $f_{y'} - f_y$: the learned logit gap (identical to standard CE).
- $\log \pi_{y'} - \log \pi_y$: fixed prior correction computed from class counts. Penalises frequent competitors; protects rare correct classes.
- No new hyperparameters, no scheduling — a single line of code.

Can also be applied post-hoc: train with standard CE, then predict $\hat{y} = \arg\max_y f_y(x) - \log \pi_y$.

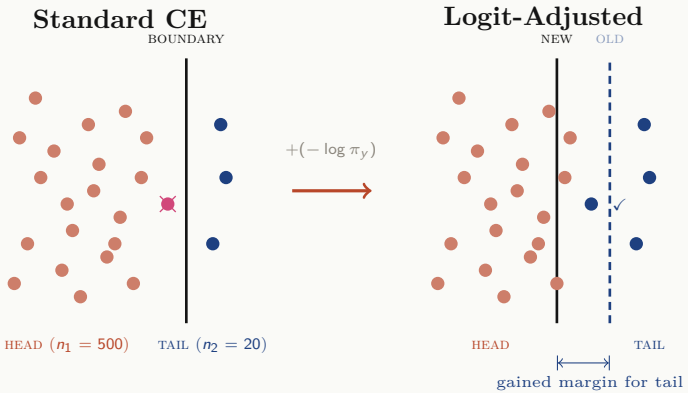
Logit Adjustment in Action

Three classes: A (head, $\pi = 0.80$), B (mid, $\pi = 0.15$), C (tail, $\pi = 0.05$). An input x truly belongs to class C.



The raw model predicts A (the head class). After subtracting $\log \pi_y$, rare classes receive a larger correction, flipping the prediction to the correct tail class C.

How Logit Adjustment Shifts the Decision Boundary



Standard CE places the boundary close to the tail, causing misclassifications. Logit adjustment shifts it toward the head, giving rare classes a wider margin without retraining.

Tale *6*

Routing Diverse Experts.

Instead of one model struggling across the entire distribution, let specialised experts handle different parts of it.

RIDE at a Glance

Wang et al. [13] observe that prior methods (cRT, τ-norm, LDAM) play a zero-sum game: every gain on the tail costs accuracy on the head.

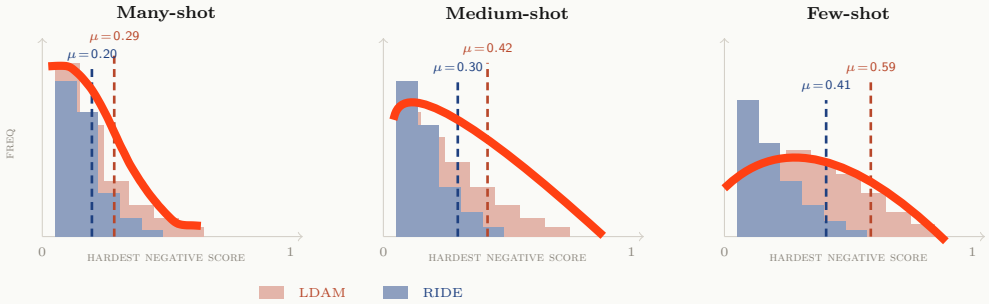
Viewing $\mathcal{D}$ as a random variable, the expected error decomposes as:

$$\text{Error}(x;\, h) = \underbrace{\text{Bias}}_{\text{accuracy}} + \underbrace{\text{Variance}}_{\text{stability}} + \underbrace{\text{Irred. Error}}_{\text{noise}}$$

- Existing methods achieve higher tail accuracy by overfitting to few tail samples, increasing variance across all splits.
- RIDE deploys multiple expert classifiers on a shared backbone, each specialising in different regions, with a routing module that dynamically assigns inputs to the most competent experts.

Hardest Negative: LDAM vs. RIDE

μ = average hardest-negative score across all instances in each split. Lower μ means the model is less confused by competing classes.



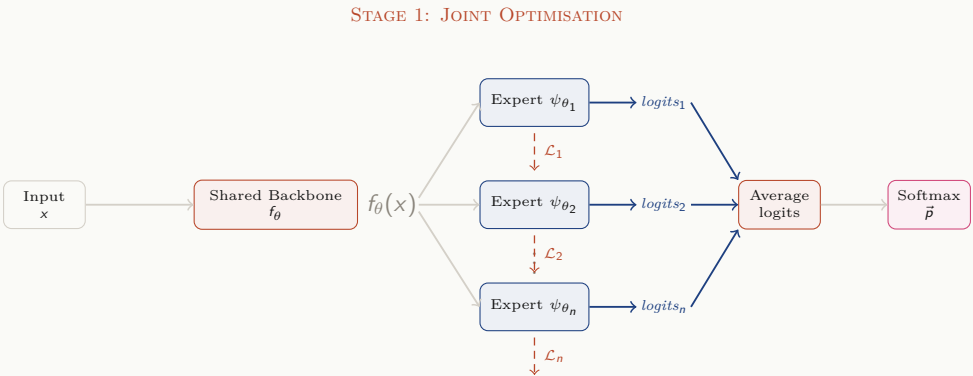
RIDE Architecture & Training

Shared backbone f_θ with n independent experts $\psi_{\theta_1}, \ldots, \psi_{\theta_n}$. Each expert is trained individually (not on the averaged output) plus a diversity loss:

$$\mathcal{L}_{\text{total}} = \sum_{i=1}^{n} \Big(\mathcal{L}_{\text{classify}}(\psi_{\theta_i}(f_\theta(x)),\, y) + \lambda\, \mathcal{L}_{\text{D-Div}}(\theta_i) \Big)$$

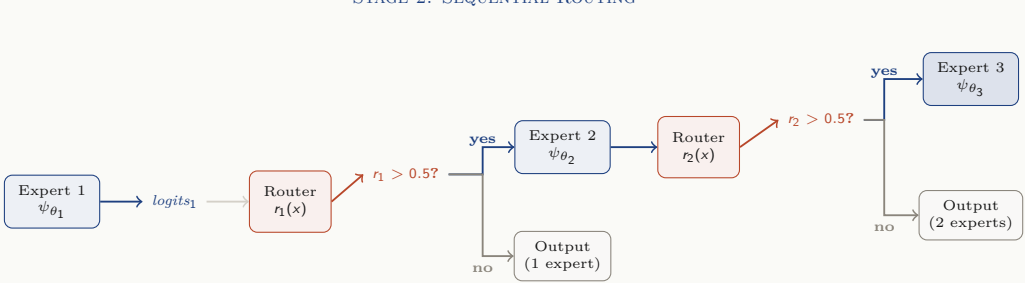
- $\mathcal{L}_{\text{D-Div}}$ maximises KL divergence between expert pairs, forcing complementary behaviour.
- At inference, active experts are averaged in logit space: $\vec{p} = \text{softmax}\big(\frac{1}{m} \sum_i \psi_{\theta_i}(f_\theta(x))\big)$.
- $\mathcal{L}_{\text{classify}}$ can be any loss (CE, LDAM, Focal) — RIDE is loss-agnostic.

RIDE Architecture: Stage 1



Each expert receives its own loss (dashed) plus a diversity loss pushing experts apart. The shared backbone f_θ receives gradients from all experts.

RIDE Architecture: Stage 2 — Dynamic Routing



Easy inputs exit early with one expert. Ambiguous inputs recruit additional experts, trading compute for accuracy only when needed.

Tale 7

Learning by Comparison.

Teach the model what makes two examples similar — and what makes them different — without ever looking at the labels.

The Core Intuition

Rather than mapping inputs to fixed class labels, contrastive learning shapes the representation space directly: pull together similar examples (positives), push apart different examples (negatives).

The result is an embedding space where proximity encodes semantic similarity, learned from the structure of the data. This is powerful for long-tailed settings: features do not depend on class frequencies, so tail classes benefit from representations shaped by the entire dataset.

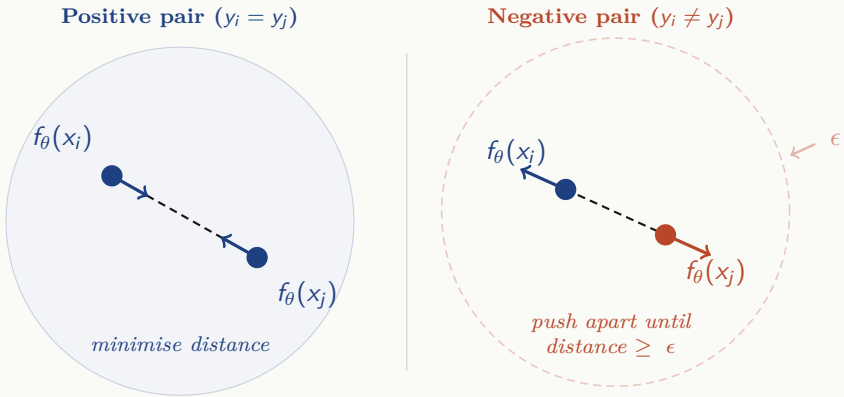
The key trend: how to incorporate multiple positives and negatives in a single batch. We trace the evolution from pairs $\rightarrow$ triplets $\rightarrow$ full-batch objectives.

Contrastive Loss

Operates on pairs (x_i, x_j) [4]:

$$\mathcal{L}_{\text{cont}} = \underbrace{\mathbb{1}[y_i = y_j]\ \|f(x_i) - f(x_j)\|_2^2}_{\text{pull same-class together}} + \underbrace{\mathbb{1}[y_i \neq y_j]\ \max(0,\ \epsilon - \|f(x_i) - f(x_j)\|_2^2)}_{\text{push different-class apart}}$$

Contrastive Loss: Illustration



Same-class embeddings are pulled together (left); different-class embeddings are pushed apart until they exceed the margin ϵ (right).

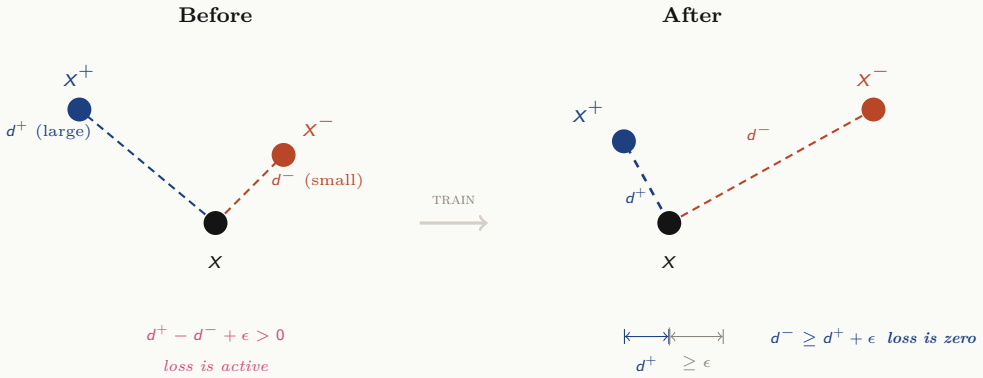
Triplet Loss

Operates on triplets (x, x^+, x^-) [10]: anchor, same-class positive, different-class negative.

$$\mathcal{L}_{\text{triplet}} = \sum_x \max\Big(0,\ \underbrace{\|f(x) - f(x^+)\|_2^2}_{\text{pos. distance}} - \underbrace{\|f(x) - f(x^-)\|_2^2}_{\text{neg. distance}} + \epsilon\Big)$$

The margin ϵ enforces that the negative must be at least ϵ farther than the positive. Selecting hard negatives is crucial.

Triplet Loss: Illustration



Before training, the negative is closer than the positive. After training, the positive is pulled close to the anchor and the negative is pushed beyond the margin ϵ.

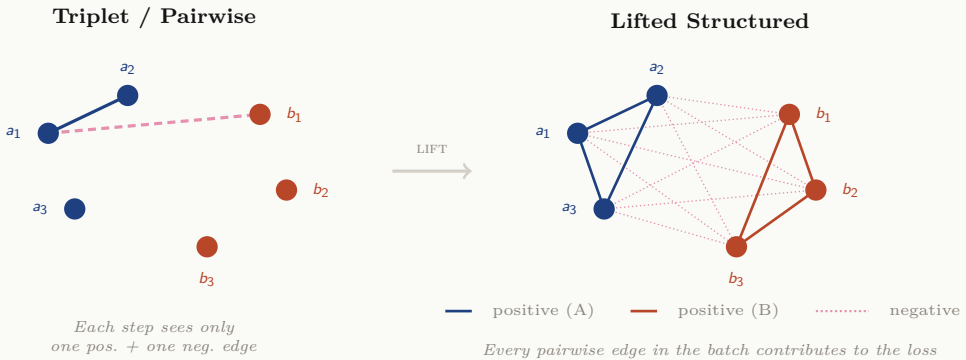
Lifted Structured Loss

Oh Song et al. [9] "lift" isolated pairs into a dense graph over all pairwise edges in the batch. For each positive pair (i, j), the smooth relaxation is:

$$\mathcal{L}_{\text{struct}}^{(i,j)} = D_{ij} + \log\Big(\sum_{(i,k)\in\mathcal{N}} e^{\epsilon - D_{ik}} + \sum_{(j,l)\in\mathcal{N}} e^{\epsilon - D_{jl}} \Big)$$

D_{ij} is the positive distance to minimise; the log-sum-exp mines the hardest negatives for both endpoints simultaneously.

Lifted Structured Loss: Illustration



Triplet loss uses a single positive and negative edge per step. The lifted structured loss considers every pairwise edge in the batch, mining the hardest violations across all samples simultaneously.

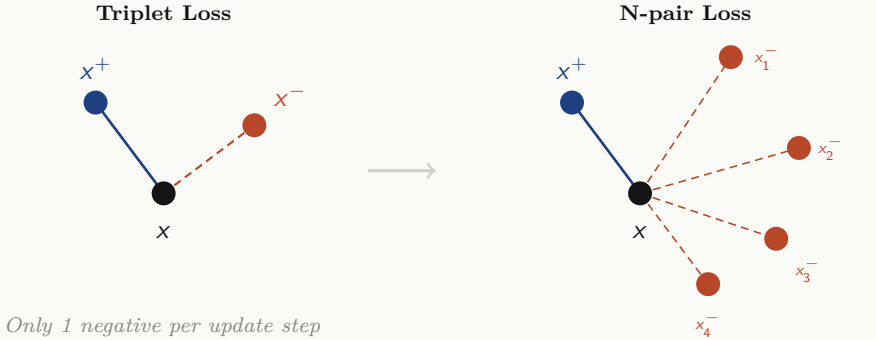
N-pair Loss

Sohn [11] generalise triplet loss to $N-1$ negatives from different classes in a single batch:

$$\mathcal{L}_{\text{N-pair}} = -\log \frac{\exp(f(x)^\top f(x^+))}{\exp(f(x)^\top f(x^+)) + \sum_{i=1}^{N-1} \exp(f(x)^\top f(x_i^-))}$$

Uses dot-product similarity; each update compares against $N-1$ negatives instead of just one. Direct precursor to InfoNCE.

N-pair Loss: Illustration



Only 1 negative per update step

$N-1$ *negatives from different classes*

Triplet loss compares against a single negative. N-pair loss compares the anchor against $N-1$ *negatives from different classes simultaneously.*

InfoNCE

From Contrastive Predictive Coding [12]. The theoretical backbone of modern contrastive learning:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)}$$

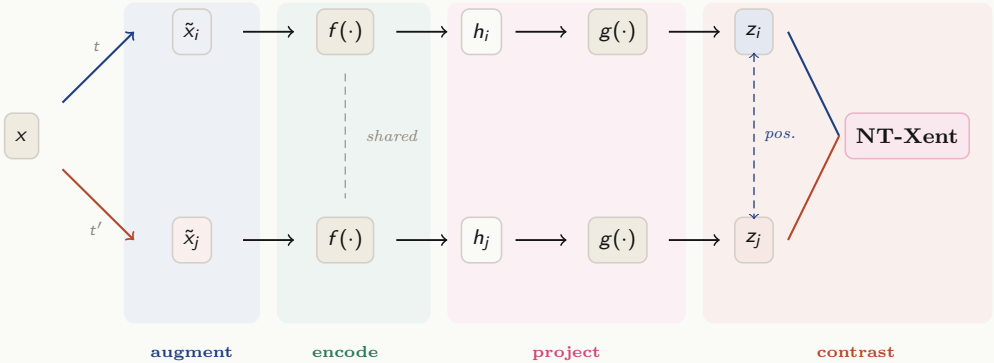
- Temperature τ controls sharpness: low τ focuses on hardest negatives; high τ treats all negatives more equally.
- Minimising InfoNCE maximises a lower bound on the mutual information $I(z_i; z_j)$.
- Forms the basis for SimCLR's NT-Xent loss.

SimCLR

Chen et al. [3]: a simple self-supervised framework built on data augmentation + encoder + projection head + NT-Xent (InfoNCE with cosine similarity).

From a batch of N images, augmentation creates $2N$ views. Each anchor has 1 positive (its augmented view) and $2(N-1)$ negatives (all other views).

SimCLR: The Framework



Two augmented views of the same image form a positive pair. All other views in the batch serve as negatives. Encoder f and projection head g are shared.

SimCLR: NT-Xent Loss

The Normalised Temperature-scaled Cross-Entropy loss for a positive pair (i, j):

$$\mathcal{L}_{\text{NT-Xent}}^{(i,j)} = -\log \frac{\exp(\text{sim}(z_i, z_j)\,/\,\tau)}{\sum_{k=1}^{2N} \mathbb{1}[k \neq i]\ \exp(\text{sim}(z_i, z_k)\,/\,\tau)}$$

- $\text{sim}(z_i, z_j) = z_i^\top z_j \,/\, (\|z_i\|\,\|z_j\|)$ — cosine similarity.
- Each anchor has 1 positive and $2(N-1)$ negatives.
- This is exactly InfoNCE with cosine similarity.
- The final loss averages over all $2N$ anchors.

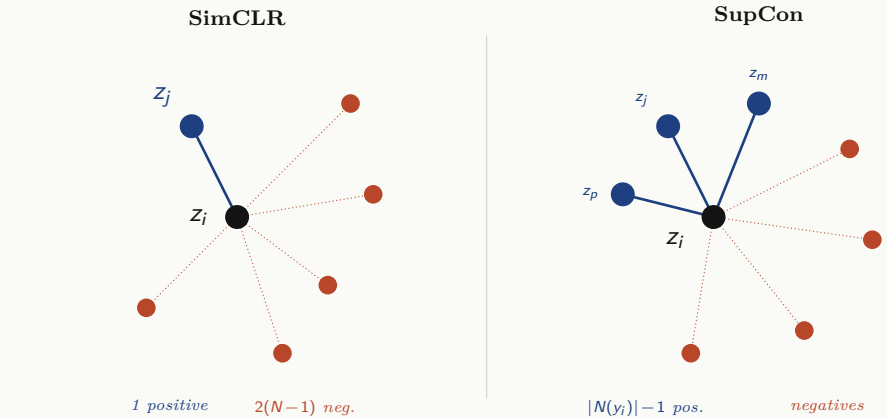
Supervised Contrastive Loss

Khosla et al. [5] extend SimCLR: with labels, all same-class samples serve as positives (not just one augmented view).

$$\mathcal{L}_{\text{SupCon}} = \sum_{i=1}^{2N} \frac{-1}{2|N(y_i)| - 1} \sum_{\substack{j \in N(y_i) \\ j \neq i}} \log \frac{\exp(z_i \cdot z_j / \tau)}{\sum_{k \neq i} \exp(z_i \cdot z_k / \tau)}$$

Under long-tailed data, head classes contribute far more positives per batch, so the contrastive signal is dominated by head classes — mirroring gradient starvation in ERM.

SupCon: Illustration



SimCLR uses only the augmented view as a positive. SupCon treats all same-class samples as positives.

Tale *8*

Balancing the Contrastive Signal.

Supervised contrastive learning inherits the long-tail bias. Two corrections: I. class-averaging and II. class-complement both to restore the balance.

BCL at a Glance

Balanced Contrastive Learning (BCL) [14] identifies that standard SupCon fails to form a regular simplex under long-tailed data. The denominator $A(i)$ is dominated by head-class samples:

- A head class may contribute $\sim$20 negatives per batch; a tail class $\sim$1.
- The gradient is biased toward separating from head classes.

BCL fixes this with two mechanisms:

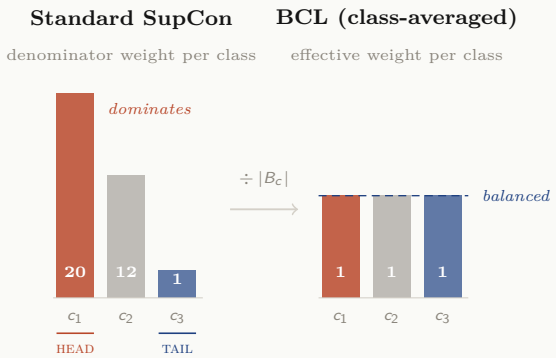
- Class-averaging: divides each negative contribution by the number of samples of that class in the batch.
- Class-complement: appends a learnable prototype μ_c for every class, guaranteeing all classes appear in every mini-batch.

I. Class-Averaging

Standard SupCon gives every sample equal weight in the denominator. Class-averaging divides by $|B_c|$:

$$\text{Standard: } \underbrace{e^{s_1} + \cdots + e^{s_{20}}}_{\text{head: 20 terms}} + \underbrace{e^{s_{21}}}_{\text{tail: 1 term}} \quad \rightarrow \quad \text{BCL: } \underbrace{\frac{e^{s_1} + \cdots + e^{s_{20}}}{20}}_{\text{1 effective term}} + \underbrace{\frac{e^{s_{21}}}{1}}_{\text{1 effective term}}$$

I. Class-Averaging: Illustration



Standard SupCon lets head classes dominate the denominator. Class-averaging normalises each class to one effective unit.

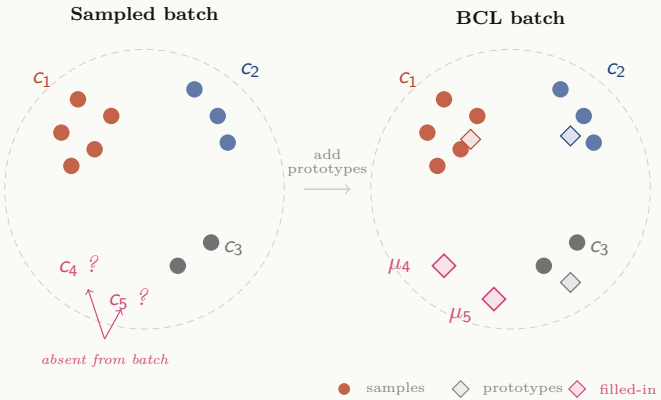
II. Class-Complement

Even with class-averaging, a tail class absent from a mini-batch contributes nothing. BCL maintains a learnable prototype μ_c for every class and appends all k prototypes to the batch:

$$Z_{\text{batch}} = \big[\, \underbrace{z_1,\ \ldots,\ z_{2B}}_{\text{augmented views}} \ ,\ \underbrace{\mu_1,\ \ldots,\ \mu_k}_{\text{class prototypes}} \,\big]$$

Prototypes are updated via backpropagation, ensuring every class exerts contrastive pressure in every batch.

II. Class-Complement: Illustration



A sampled batch may miss tail classes entirely. BCL appends learnable prototypes so every class is always represented.

The BCL Loss & Training Pipeline

$$\mathcal{L}_{\text{BCL}}^{(i)} = -\frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{e^{z_i \cdot z_p / \tau}}{\sum_{a \in \tilde{A}(i)} \frac{e^{z_i \cdot z_a / \tau}}{|B_{y_a}|}}$$

$\tilde{A}(i)$ includes batch samples + prototypes (class-complement); $|B_{y_a}|$ normalises per class (class-averaging). BCL trains end-to-end combining both objectives:

$$\mathcal{L} = \underbrace{\alpha\, \mathcal{L}_{\text{LA}}}_{\text{Logit-Adjusted CE}} + \underbrace{\beta\, \mathcal{L}_{\text{BCL}}}_{\text{Balanced Contrastive}}$$

Each image produces 3 views: 1 for classification, 2 for contrastive. Defaults: $\alpha = 1$, $\beta = 0.35$, $\tau = 0.07$.

Tale *9*

A Tale of Two Classes.

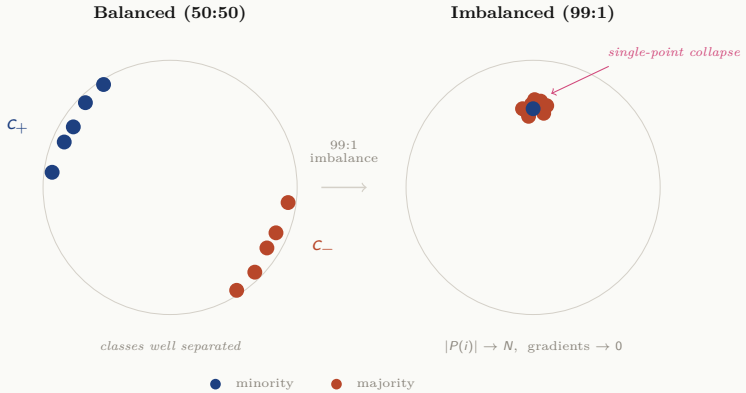
Supervised contrastive learning collapses under binary imbalance. Two targeted fixes: SupMin and SupConProto restore separation when only two classes exist.

The Binary Collapse Problem

Mildenberger et al. [8]: under binary imbalance (e.g. 99:1), the majority floods each batch. The SupCon gradient magnitude is upper-bounded by $1/|P(i)|$. As $|P(i)| \to N$, gradients vanish and all embeddings collapse to a single point on the hypersphere.

- This is a binary-specific gradient saturation failure, distinct from the multi-class simplex distortion BCL addresses.
- Canonical metrics (SAD, CAD, GPU) fail to detect collapse; the novel SAA and CAC do.

The Binary Collapse Problem: Illustration



Under balanced sampling both classes occupy distinct regions. Under extreme imbalance all embeddings collapse to a single point — the minority class becomes indistinguishable.

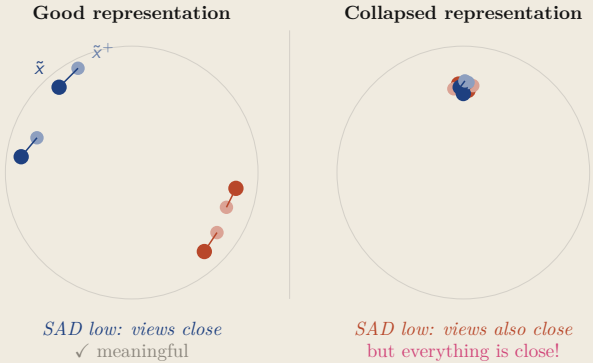
I. Sample Alignment Distance (SAD)

$$\mathsf{SAD} = \frac{1}{|X|} \sum_{x \in X} \| f(\tilde{x}) - f(\tilde{x}^{+}) \|_2$$

Measures how close two augmented views of the same sample are. Lower is better.

Failure mode: under collapse, all distances are near zero — SAD reports a perfect score but never checks whether different samples are far.

I. SAD — Illustration



Left: *augmented views are close and classes are separated (SAD is* ***low*** *and meaningful).*
Right: *everything collapsed (SAD is also* ***low****), but only because all distances are near zero.*

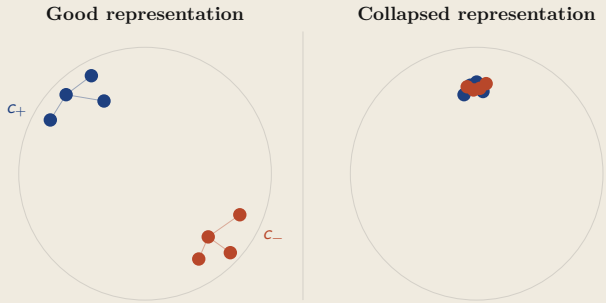
II. Class Alignment Distance (CAD)

$$\mathrm{CAD} = \frac{1}{C} \sum_{c=1}^{C} \frac{1}{\binom{|X_c|}{2}} \sum_{\tilde{x}, \tilde{x}' \in W_c} \left\| f(\tilde{x}) - f(\tilde{x}') \right\|_2$$

Measures pairwise distances within each class. Lower means tighter clusters.

Failure mode: under collapse, intra-class distances are trivially small. CAD never compares across classes, so it cannot detect overlap.

II. CAD — Illustration



CAD low: classes tight
✓ meaningful

CAD low: classes also tight
but classes overlap!

Left: *tight clusters that are well separated (CAD is* ***low*** *and meaningful).* ***Right:*** *both classes overlap in one blob (CAD is also* ***low****), but the classes are indistinguishable.*

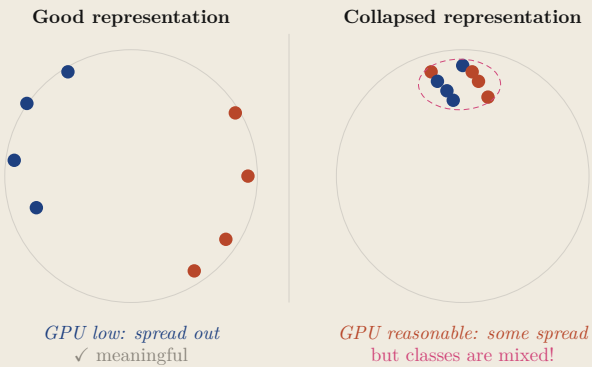
III. Gaussian Potential Uniformity (GPU)

$$\mathrm{GPU} = \log\left[\frac{1}{\binom{N}{2}} \sum_{k} \sum_{j \neq k} e^{-\|f(\tilde{x}_k) - f(\tilde{x}_j)\|_2^2}\right]$$

Measures how uniformly embeddings are spread on the hypersphere. More negative is better.

Failure mode: a collapsed blob with internal variance can still report a reasonable GPU score. GPU is blind to class boundaries.

III. GPU — Illustration



Left: *embeddings spread with class structure preserved (GPU is* ***low*** *and meaningful).* ***Right:*** *a mixed-class blob with internal variance (GPU is still* ***reasonable****), but the classes are completely interleaved.*

IV. Sample Alignment Accuracy (SAA)

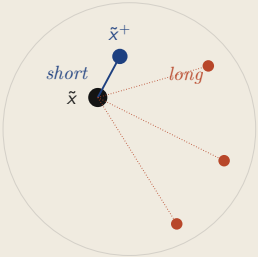
$$\mathsf{SAA} = \frac{1}{|X|} \sum_{x \in X} \mathbf{1}\Big[\big\| f(\tilde{x}) - f(\tilde{x}^+) \big\|_2 \; < \; \min_{\tilde{x}^-} \big\| f(\tilde{x}) - f(\tilde{x}^-) \big\|_2 \Big]$$

Asks a relative question: is each sample's positive neighbour closer than all negatives? Higher is better.

Succeeds where SAD fails: under collapse, positive and negative distances converge to the same value, so the indicator returns 0. SAA correctly detects collapse.

IV. SAA — Illustration

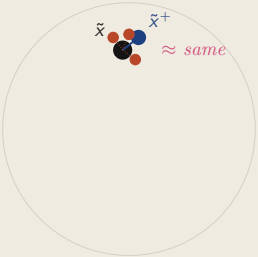
Good representation



$\|\tilde{x} - \tilde{x}^+\| < \min_{\tilde{x}^-} \|\tilde{x} - \tilde{x}^-\|$

✓ **SAA = 1 for this sample**

Collapsed representation



positive not closer than negatives

× **SAA = 0 for this sample**

Left: *the positive neighbour is clearly closer than any negative (SAA counts this sample as* ***correct****).* ***Right:*** *under collapse, positive and negatives are at the same distance (SAA correctly* ***flags the failure****).*

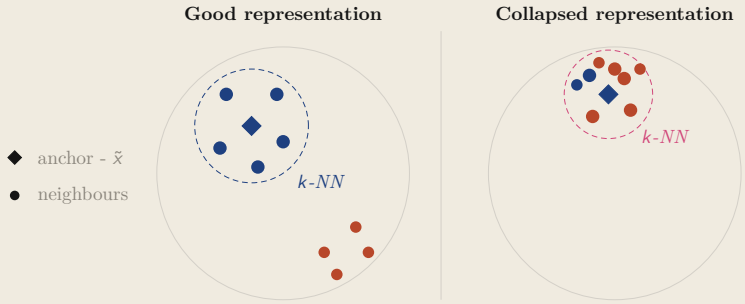
V. Class Alignment Consistency (CAC)

$$\mathsf{CAC} = \frac{1}{|D|} \sum_{(\tilde{x},\, W_{\tilde{x}}) \in D} \frac{1}{|W_{\tilde{x}}|} \sum_{\tilde{x}' \in W_{\tilde{x}}} \mathbf{1}\big[\, g(\tilde{x}) = g(\tilde{x}') \,\big]$$

Measures neighbourhood purity: what fraction of each sample's k-NN share its class? Higher is better.

Succeeds where GPU fails: under collapse, minority neighbourhoods are flooded by majority samples. CAC achieves $R^2 = 0.69$ correlation with downstream accuracy, the strongest among all five metrics.

V. CAC — Illustration



Left: *the k-NN neighbourhood is class-pure (CAC =* ***1.0****).* ***Right:*** *under collapse, the neighbourhood is dominated by the majority class (CAC =* ***0.2*** *for this minority sample).*

Method I: SupMin

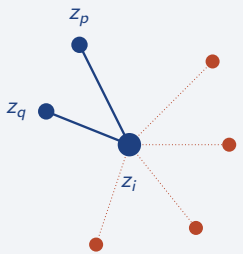
Gradient saturation affects only the majority class. SupMin exploits this asymmetry:

$$\mathcal{L}_{\text{SupMin}} = \underbrace{\mathcal{L}^{\text{min}}_{\text{SupCon}}}_{\substack{\text{minority: labels used,} \\ \text{few positives, strong gradients}}} + \underbrace{\mathcal{L}^{\text{maj}}_{\text{NT-Xent}}}_{\substack{\text{majority: self-supervised,} \\ |P(i)| = 1\text{, no saturation}}}$$

No new hyperparameters. Labels are dropped only where they cause harm (majority anchors with $|P(i)| \to N$), and kept where they help (minority anchors with few positives).

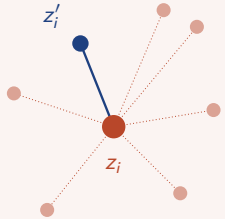
Method I: SupMin — Illustration

Minority anchors: SupCon



labels used: few positives
⇒ strong gradients

Majority anchors: NT-Xent



no labels: only augment is positive
⇒ no saturation

***Minority anchors** keep supervised contrastive edges (few positives, strong gradients).*
***Majority anchors** switch to self-supervised NT-Xent (one augmented positive, no saturation).*

Method II: SupConProto

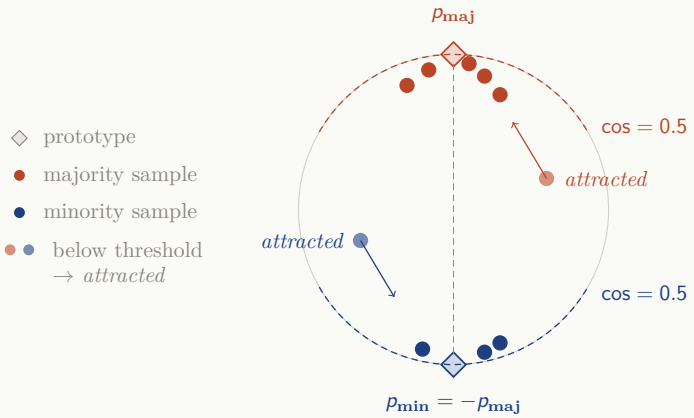
An alternative using fixed geometric targets:

- Place two antipodal prototypes on the hypersphere: p_{maj} (learnable) and $p_{\text{min}} = -p_{\text{maj}}$.
- Each sample is attracted toward its class prototype, but only when $\cos(z_i, p_{y_i}) \leq 0.5$. Once close enough, only NT-Xent applies — preserving within-class spread.

$$\mathcal{L}_{\text{SupConProto}} = \mathcal{L}_{\text{proto}} + \mathcal{L}_{\text{NT-Xent}}$$

Self-regulating: antipodal prototypes enforce inter-class separation; the threshold + NT-Xent preserves intra-class uniformity.

Method II: SupConProto — Illustration



Two antipodal prototypes define the target geometry. Samples beyond the cos = 0.5 *threshold are pulled toward their prototype; once close enough, only NT-Xent applies.*

A Tale of Two Classes: Summary

Standard SupCon collapses under binary imbalance. Canonical metrics (SAD, CAD, GPU) fail to detect this; the novel SAA and CAC do.

Two independent methods (alternatives, not combined):

- SupMin — splits the loss by class: SupCon where gradients are healthy (minority), NT-Xent where they saturate (majority).
- SupConProto — a unified loss for all samples: fixed antipodal prototypes + cosine threshold + NT-Xent.

Both methods restore class separation under extreme binary imbalance.

References I

[1] Kaidi Cao, Colin Wei, Adrien Gaidon, Nikos Arechiga, and Tengyu Ma. Learning imbalanced datasets with label-distribution-aware margin loss. *Advances in Neural Information Processing Systems*, 32, 2019.

[2] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[3] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning*, pages 1597–1607, 2020.

[4] Sumit Chopra, Raia Hadsell, and Yann LeCun. Learning a similarity metric discriminatively, with application to face verification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 539–546, 2005.

[5] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 18661–18673, 2020.

[6] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017.

[7] Aditya Krishna Menon, Sadeep Jayasumana, Ankit Singh Rawat, Himanshu Jain, Andreas Veit, and Sanjiv Kumar. Long-tail learning via logit adjustment. *arXiv preprint arXiv:2007.07314*, 2020.

References II

[8] David Mildenberger, Paul Hager, Daniel Rueckert, and Martin J. Menten. A tale of two classes: Adapting supervised contrastive learning to binary imbalanced datasets. *arXiv preprint arXiv:2503.17024*, 2025.

[9] Hyun Oh Song, Yu Xiang, Stefanie Jegelka, and Silvio Savarese. Deep metric learning via lifted structured feature embedding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4004–4012, 2016.

[10] Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 815–823, 2015.

[11] Kihyuk Sohn. Improved deep metric learning with multi-class n-pair loss objective. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[12] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

[13] Xudong Wang, Long Lian, Zhongqi Miao, Ziwei Liu, and Stella X. Yu. Long-tailed recognition by routing diverse distribution-aware experts. In *International Conference on Learning Representations*, 2021.

[14] Jianggang Zhu, Zheng Wang, Jingjing Chen, Yi-Ping Phoebe Chen, and Yu-Gang Jiang. Balanced contrastive learning for long-tailed visual recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6908–6917, 2022.